Danh sách GitHub repo hỗ trợ tiết kiệm token cho LLM

Cập nhật: 28-10-2026

Phạm vi: shortlist các repo công khai có thể giảm token đầu vào, tránh gọi LLM lặp lại, rút gọn lịch sử hội thoại hoặc giúp đo và tối ưu mức sử dụng.

Với prompt tiếng Việt: đây là các công cụ LLM dùng chung, không phải benchmark riêng cho tiếng Việt. Hãy kiểm tra tỷ lệ token và chất lượng trả lời trên chính bộ prompt tiếng Việt của bạn.

Cách đọc phần trăm

- Phần trăm trong bảng là kết quả benchmark/giới hạn được nguồn công bố, hoặc giá trị phụ thuộc workload; không phải mức tiết kiệm bảo đảm khi cài repo.
- Với mức nén **N×**, phần token đầu vào giảm là **1 - 1/N**. Ví dụ **20×** tương ứng tối đa **95%** ít token đầu vào hơn.
- Các con số của LLMLingua và Mem0 thuộc những benchmark cụ thể. Tokenizer, ngôn ngữ, loại prompt, độ dài câu trả lời và yêu cầu chất lượng làm kết quả thực tế thay đổi.
- **N/A** nghĩa là nguồn chính thức không công bố một tỷ lệ tiết kiệm token chung cho mọi dự án. **Không tự giảm token** nghĩa là repo chủ yếu đo lường hoặc tối ưu hạ tầng và không tự rút bớt prompt/output.
- Bảng tách **token** khỏi **chi phí**: dùng model rẻ hơn có thể giảm tiền nhưng không đồng nghĩa với giảm token.

1. Nén prompt và context

Đây là nhóm có tác động trực tiếp và đo được nhất lên **input tokens**. Có thể dùng một biến thể riêng hoặc tích hợp vào pipeline RAG.

Repo / biến thể	Phương pháp	% token giảm được	Căn cứ và giới hạn
microsoft/LLMLingua — bản gốc	Xếp hạng và loại token ít quan trọng trong prompt	**Tối đa 95%**	Repo báo cáo nén tối đa **20×**; con số là mức tối đa trên tác vụ thử nghiệm, không phải mặc định.
microsoft/LLMLingua — LongLLMLingua	Nén context dài có xét câu hỏi, ưu tiên đoạn liên quan	**Khoảng 50–83%**	Bài báo thử mức nén **2×–6×** trên prompt dài khoảng 10k token; riêng NaturalQuestions báo cáo gần **4×** ít token hơn (khoảng **75%**). Benchmark-specific; chất lượng và kết quả tùy tác vụ.
microsoft/LLMLingua — LLMLingua-2	Nén trích xuất dựa trên phân loại token	**Khoảng 50–80%**	Bài báo báo cáo compression ratio **2×–5×**; tỷ lệ token giảm được quy đổi từ các mức nén đó.

2. Memory cho hội thoại dài

Repo	Phương pháp	Tỷ lệ theo nguồn	Căn cứ và giới hạn
mem0ai/mem0	Lưu thông tin quan trọng thành memory rồi truy xuất theo câu hỏi, thay vì gửi toàn bộ lịch sử	**Trên 90% token cost**	Đây là mức tiết kiệm chi phí token được nhóm tác giả báo cáo so với full-context trên LOCOMO; không phải phần trăm token-count reduction chung. Benchmark không bảo đảm cùng kết quả cho hội thoại khác.
getzep/graphiti	Knowledge graph theo thời gian, truy xuất sự kiện và quan hệ phù hợp	**N/A — cần benchmark**	Có thể tránh gửi cả lịch sử, nhưng repo không đưa ra một tỷ lệ token giảm chung. Tính cả chi phí tạo/cập nhật graph khi so sánh.

3. Retrieval, RAG và lọc tài liệu

Các repo này giúp lấy phần dữ liệu phù hợp thay vì chèn toàn bộ kho tài liệu vào prompt. Mức tiết kiệm do cấu hình retrieval, số chunk, kích thước chunk và độ dài tài liệu quyết định.

Repo	Phương pháp	% token giảm được	Căn cứ và giới hạn
run-llama/llama_index	Index, retrieval, reranking và tối ưu context RAG	**N/A — cần benchmark**	Framework hỗ trợ lọc context; không có tỷ lệ chung cho mọi index và **top-k**.
lancedb/lancedb	Vector/multimodal database cho retrieval	**N/A — cần benchmark**	Có thể chỉ lấy các đoạn liên quan; cần đo số token sau khi render context.
stanford-futuredata/ColBERT	Neural retrieval để tìm passage phù hợp	**N/A — cần benchmark**	Chất lượng recall và số passage gửi vào prompt quyết định kết quả.
neuml/txtai	Semantic search và pipeline retrieval	**N/A — cần benchmark**	Repo không cam kết một phần trăm tiết kiệm token tổng quát.
Unstructured-IO/unstructured	Trích xuất, chuẩn hóa và chia nhỏ tài liệu	**N/A — cần benchmark**	Có thể lọc phần dư thừa, nhưng không bảo đảm giảm token; kết quả phụ thuộc tài liệu, metadata và cấu hình chunking.
sinaptik-ai/pandas-ai	Tạo truy vấn trên dữ liệu có cấu trúc thay vì gửi cả bảng	**N/A — cần benchmark**	Có thể tránh đưa cả bảng vào context; thực tế phụ thuộc schema, câu hỏi và SQL/Python được tạo.

4. Semantic cache

Repo	Phương pháp	% token giảm được	Cách tính đúng
zilliztech/GPTCache	Lưu câu trả lời; truy vấn mới tương tự có thể được trả từ cache	**0–100% theo lượng token cache hit**	Mỗi cache hit có thể tránh gần như toàn bộ input/output token của lần gọi LLM bị bỏ qua. Tỷ lệ chung phải tính theo tổng token baseline của các hit, không chỉ theo số request; chưa tính token/chi phí của embedding và tra cứu cache.

5. Routing và gateway

Nhóm này chủ yếu tối ưu **chi phí hoặc lựa chọn model**. Chọn model rẻ hơn không tự động làm số token ít đi; input/output token có thể thay đổi theo model và chất lượng câu trả lời.

Repo	Phương pháp	% token giảm được	Tác động chính
BerriAI/litellm	Gateway, fallback, routing và theo dõi chi phí	**N/A — không có mức chung**	Có thể giảm chi phí bằng cách route tới model rẻ hơn; không bảo đảm giảm token.
Portkey-AI/gateway	Gateway đa provider, policy và quản lý request	**N/A — không có mức chung**	Tối ưu vận hành/provider; đo token và chi phí riêng.
aurelio-labs/semantic-router	Phân loại ý định để chọn model/workflow	**N/A — không có mức chung**	Có thể tránh dùng model lớn cho request đơn giản; phần giảm chủ yếu là chi phí.

6. Quan sát và đánh giá

Các repo dưới đây giúp tìm prompt dài, request lặp hoặc model đắt; bản thân công cụ quan sát/đánh giá không cắt token cho đến khi có người sửa pipeline.

Repo	Vai trò	% token giảm trực tiếp	Ghi chú
langfuse/langfuse	Trace, đánh giá và theo dõi token/chi phí	**Không tự giảm token**	Hỗ trợ tìm chỗ lãng phí để tối ưu sau đó.
comet-ml/opik	Tracing và đánh giá ứng dụng LLM	**Không tự giảm token**	Không thay đổi prompt hoặc output nếu chưa cấu hình tối ưu.
Helicone/helicone	Theo dõi request, usage và chi phí	**Không tự giảm token**	Công cụ đo lường; không có phần trăm tiết kiệm mặc định.
Arize-ai/phoenix	Quan sát trace, retrieval và chất lượng	**Không tự giảm token**	Hỗ trợ chẩn đoán RAG; token giảm phụ thuộc thay đổi sau đánh giá.
openlit/openlit	OpenTelemetry cho usage, độ trễ và chi phí LLM	**Không tự giảm token**	Telemetry không tự rút gọn context.
microsoft/promptflow	Xây, chạy và đánh giá flow LLM	**N/A — cần benchmark**	Flow có thể được tối ưu; kết quả phụ thuộc thiết kế.
promptfoo/promptfoo	So sánh prompt/model và kiểm tra chất lượng	**Không tự giảm token**	Hỗ trợ chọn cấu hình đạt chất lượng với ít chi phí hơn.
stanford-crfm/helm	Benchmark model theo nhiều tiêu chí	**Không tự giảm token**	Hỗ trợ chọn model; không trực tiếp giảm token.

7. Tối ưu prompt, output và cấu trúc

Repo	Phương pháp	% token giảm được	Căn cứ và giới hạn
stanfordnlp/dspy	Tối ưu chương trình/prompt theo metric	**N/A — cần benchmark**	Chất lượng có thể tăng trên mỗi token; mức giảm token tùy module và dữ liệu.
dottxt-ai/outlines	Ràng buộc output theo schema/grammar	**N/A — cần benchmark**	Có thể giảm output sai định dạng và lần gọi lại; không có mức giảm chung.
567-labs/instructor	Output có cấu trúc và xác thực schema	**N/A — cần benchmark**	Số lần retry có thể giảm; đo cả token của lần thử lỗi và validation.
microsoft/semantic-kernel	Orchestration, plugin và quản lý memory	**N/A — cần benchmark**	Framework không tự động giảm context; tùy cách triển khai.

8. Local inference và serving

Nhóm này có thể giảm chi phí hạ tầng hoặc tăng thông lượng. Với cùng prompt và cách sinh, nó **không trực tiếp làm giảm số token**.

Repo	Phương pháp	% token giảm trực tiếp	Lợi ích chính
ggml-org/llama.cpp	Chạy model local/quantized	**Không tự giảm token**	Có thể giảm chi phí API/hạ tầng; token phụ thuộc prompt và model.
vllm-project/vllm	Serving throughput cao và quản lý bộ nhớ	**Không tự giảm token**	Tăng hiệu suất GPU/requests, không tự rút prompt.
sgl-project/sglang	Runtime serving và tối ưu thực thi	**Không tự giảm token**	Tối ưu latency/throughput, không phải token compression.
lm-sys/FastChat	Serving, huấn luyện và đánh giá model mở	**Không tự giảm token**	Tự host có thể giảm chi phí API; không có phần trăm token giảm trực tiếp.

Gợi ý chọn nhanh

- **Muốn giảm input token ngay:** thử LLMLingua-2 hoặc LongLLMLingua trên tập prompt đại diện; bắt đầu ở mức nén vừa rồi kiểm tra độ chính xác.
- **Agent có lịch sử dài:** so sánh Mem0 với full-context trên hội thoại của bạn; tính cả token dùng để trích xuất/cập nhật memory.

- **Request hay lặp lại:** thử GPTCache; báo cáo tỷ lệ hit theo token và kiểm tra freshness/false hit.
- **RAG tài liệu:** bắt đầu với retrieval (ví dụ LlamaIndex + LanceDB/ColBERT), rồi mới thêm compressor nếu retrieval vẫn đưa quá nhiều context.
- **Chỉ muốn hóa đơn thấp hơn:** gateway/routing hoặc local serving có thể phù hợp, nhưng theo dõi riêng **input_tokens**, **output_tokens**, **cost** và chất lượng.

Nguồn cho các con số

- Microsoft LLMLingua repository — mô tả nén tối đa **20×** và các biến thể.
- LongLLMLingua, ACL 2024 — báo cáo gần **4×** ít token hơn trên NaturalQuestions và thử các mức nén **2×–6×** trên prompt dài.
- LLMLingua-2, ACL 2024 Findings — báo cáo tỷ lệ nén **2×–5×**.
- Mem0 paper, arXiv — báo cáo tiết kiệm hơn **90%** token cost so với full-context trong benchmark LOCOMO.
- GPTCache repository — mô tả semantic cache và metric hit ratio; tỷ lệ thực tế do workload quyết định.

Kết luận: không có một tỷ lệ tiết kiệm chung áp dụng cho mọi repo và mọi dự án. Hãy ghi lại baseline và kết quả trên cùng một tập request; chỉ dùng con số trong bảng như benchmark tham khảo hoặc công thức tính.